

Time & Space Complexity Notes

Comprehensive Computer Science Revision Guide & Python Data Structure Analysis

1. FUNDAMENTALS OF COMPLEXITY

What is Time Complexity?

Time complexity quantifies the amount of computational time an algorithm takes to run as a function of the input length (n). It measures how execution steps scale rather than exact runtime in seconds (which varies by hardware).

Why We Use It

- **Performance Prediction:** Predicts algorithm scalability before deployment.
- **Objective Comparison:** Compares efficiency independent of machine specs or language.

What is Big-O Notation?

Big-O notation (O) describes the **upper bound** or worst-case scenario of an algorithm's growth rate. It drops low-order terms and constants to focus purely on asymptotic behavior as $n \rightarrow \infty$.

Time vs Space Complexity:

- **Time Complexity:** Operational runtime growth rate.
- **Space Complexity:** Peak auxiliary memory/storage required.

2. COMMON COMPLEXITIES & CALCULATION RULES

Notation	Name	Description & Example
$O(1)$	Constant Time	Direct memory lookup (e.g., accessing array index, hash table key).
$O(\log n)$	Logarithmic Time	Search space is halved in every step (e.g., Binary Search).
$O(n)$	Linear Time	Iterates through input once (e.g., linear search, simple loop).
$O(n \log n)$	Linearithmic Time	Standard efficiency limit for comparison sorts (e.g., Timsort, Merge Sort).
$O(n^2)$	Quadratic Time	Nested iterations over the dataset (e.g., Bubble Sort, Matrix addition).

How to Calculate Time Complexity

1. **Identify the Input Size:** Define parameter n .
2. **Count Total Steps:** Express operations as a formula $f(n)$.
3. **Keep Dominant Term:** Drop lower-order terms (e.g., $3n^2 + 5n + 12 \rightarrow O(n^2)$).
4. **Drop Constants:** Multipliers are ignored (e.g., $2n \rightarrow O(n)$).

3. STRUCTURAL MECHANICS (WHY OPERATIONS TAKE TIME)

Lists (Dynamic Arrays)

- **Indexing $O(1)$:** Contiguous memory allows direct address offset computation: $Address = Base + i \times Size$.
- **Search $O(n)$:** Must inspect elements sequentially from start to end.
- **Insert/Delete $O(n)$:** Modifying middle/front forces all subsequent elements to shift in memory.

Dictionaries & Sets (Hash Tables)

- **Key/Set Lookup $O(1)$ Avg:** Hash function converts key directly into array index location.
- **Insert/Delete $O(1)$ Avg:** Direct bucket indexing enables instant updates.
- **Worst-Case $O(n)$:** Occurs when severe hash collisions force key comparisons in a single degraded chain/bucket.

- **Append $O(1)$ Amortized** : Adding at end requires no shifts (occasional resize cost spread over many pushes).

4. COMPLETE DATA STRUCTURE COMPLEXITY TABLES

Python List []

Operation	Average	Worst / Amortized	Reason & Mechanic
Get Item (list[i])	$O(1)$	$O(1)$	Direct offset address calculation.
Set Item (list[i] = x)	$O(1)$	$O(1)$	Direct memory overwriting.
append(x)	$O(1)$	$O(1)$ Amortized	Appends to unused tail space.
pop()	$O(1)$	$O(1)$	Removes last element; no shift needed.
pop(i) / insert(i, x)	$O(n)$	$O(n)$	Requires shifting remaining elements.
x in list (Search)	$O(n)$	$O(n)$	Linear iteration over all elements.
Delete Item (del list[i])	$O(n)$	$O(n)$	Requires shifting elements back.
sort()	$O(n \log n)$	$O(n \log n)$	Adaptive Timsort algorithm.
len(list)	$O(1)$	$O(1)$	Stored length attribute on object struct.

Python Dictionary {}

Operation	Average	Worst
d[key] (Get/Set)	$O(1)$	$O(n)$
key in d (Search)	$O(1)$	$O(n)$
Delete Item	$O(1)$	$O(n)$
Iteration	$O(n)$	$O(n)$

Python Set set()

Operation	Average	Worst
x in s	$O(1)$	$O(n)$
Union ()	$O(n+m)$	$O(n+m)$
Intersection (&)	$O(\min(n,m))$	$O(n*m)$
Difference (-)	$O(n)$	$O(n*m)$

5. COMPARISON & EXAM-WINNING RULES

Side-by-Side Comparison

Operation	List []	Dictionary {}	Set set()
Search (x in container)	$O(n)$	$O(1)$ Avg	$O(1)$ Avg
Get / Lookup	$O(1)$ (by index)	$O(1)$ Avg (by key)	N/A (Unindexed)
Insert / Add	$O(1)$ (end) / $O(n)$ (mid)	$O(1)$ Avg	$O(1)$ Avg
Delete	$O(n)$	$O(1)$ Avg	$O(1)$ Avg
Length (len)	$O(1)$	$O(1)$	$O(1)$

Top Exam Tricks & Revision Cheatsheet

Memory Rule of Thumb

List: End is fast ($O(1)$), middle/front is slow ($O(n)$ due to shifts).

Dict/Set: Search is fast ($O(1)$), but lacks numeric indexing.

Code Loop Identification

- 1 Loop = $O(n)$
- 2 Nested Loops = $O(n^2)$
- Halving input step (e.g. $i = i // 2$) = $O(\log n)$